Memory

2025 Semester 1 SOFTENG 370: Operating Systems
Talia Xu

Lecture 11
1.0.0

Trivia!

We always say Python is slow, but why is `Numpy` so fast?

You are playing for: 1x cold brew (unopened version)

Trivia!

We always say Python is slow, but why is `Numpy` so fast?

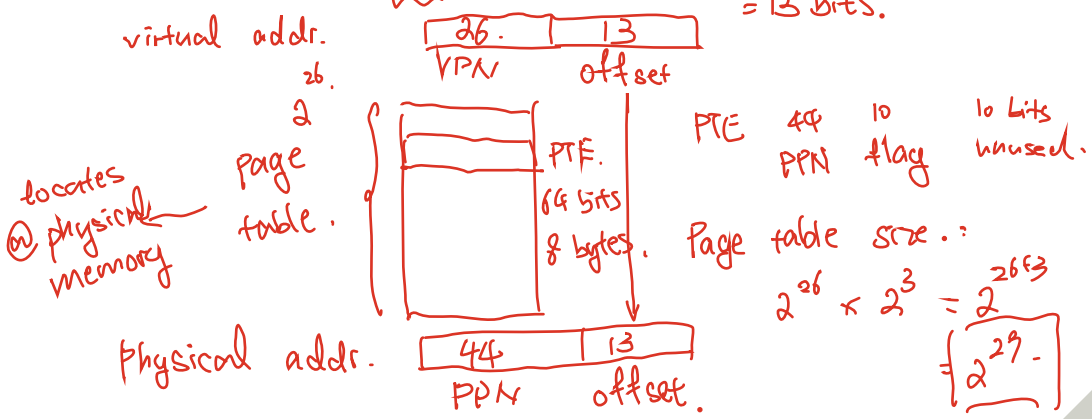
It's implemented in C!

Review: Page Table

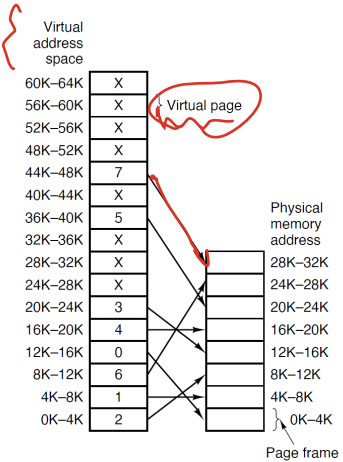
RISC-V

Let's say we have Sv39 with 8kB page size.

log(8kB) = 13 bits.



Not all virtual memory is mapped to physical memory

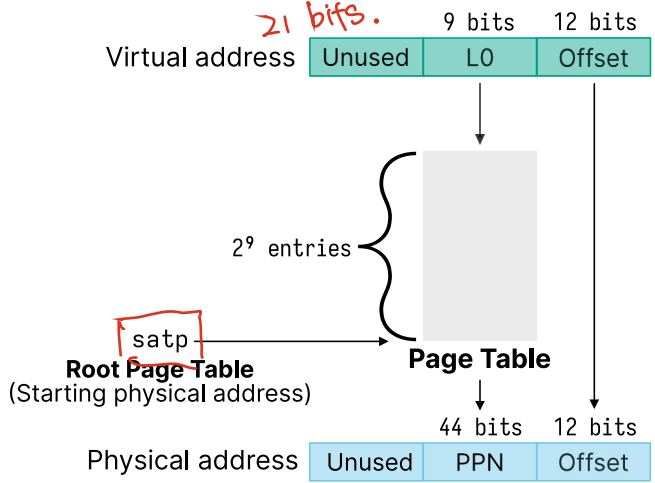


What Should We Do About the Page Table Size?

Most programs don’t use all the virtual memory space, how can we take advantage?

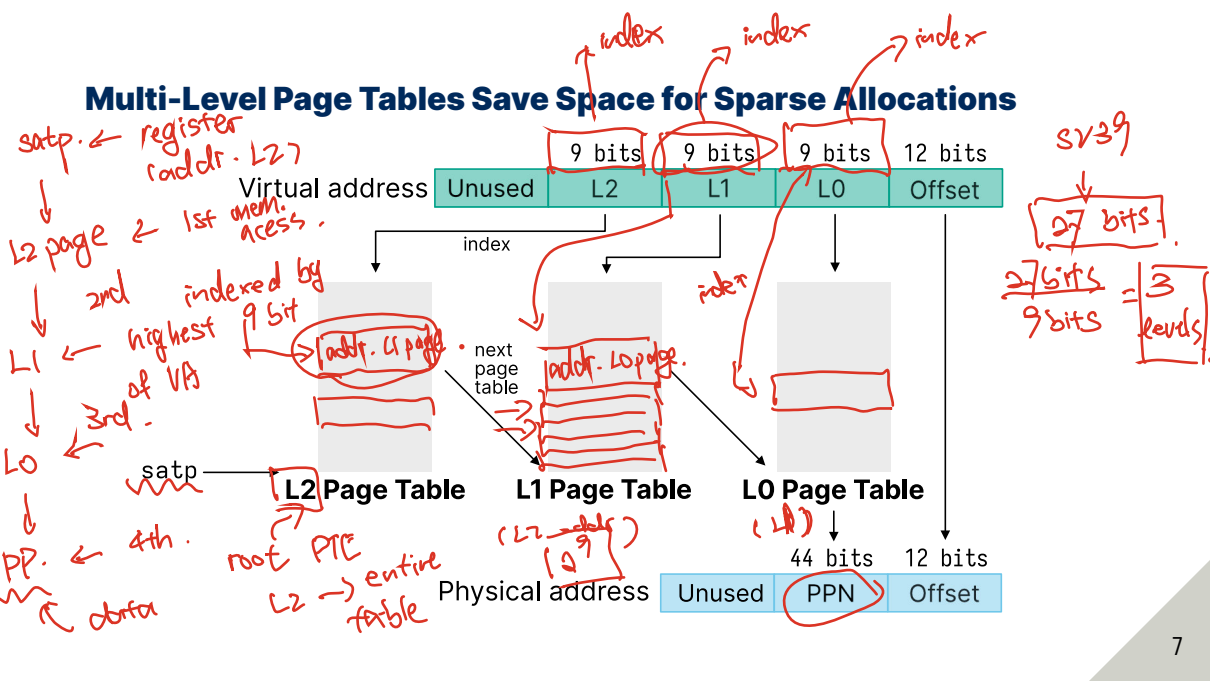
We Can Make Our Page Table Fit on a Page

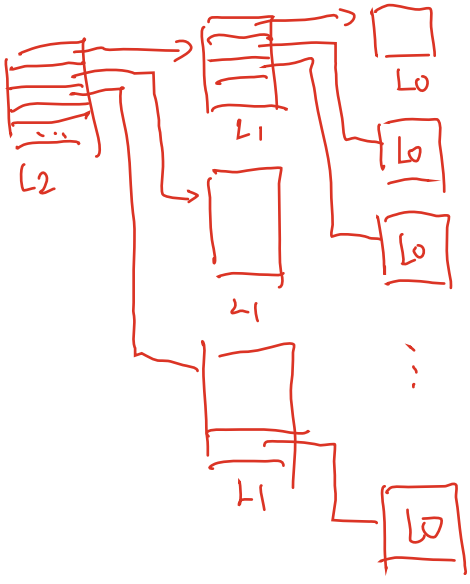
smallest unit of memory.



Page size 4KB
$= 2^{12}$.

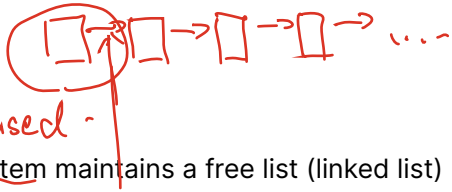
Multi-Level Page Tables Save Space for Sparse Allocations





$2^3 \times 2^3 \times 2^4$.

Page Allocation Uses A Free List



Given physical pages, the operating system maintains a free list (linked list)

The unused pages themselves contain the next pointer in the free list
- Physical memory gets initialized at boot

To allocate a page, you remove it from the free list
- To deallocate a page you add it back to the free list

Insight: Use a Page for Each Smaller Page Table

There are 512 (2^9) entries of 8 bytes(2^3) each, which is 4096 bytes

The PTE for L(N) points to the page table for L(N-1)

You follow these page tables until L0 and that contains the `PPN`

Consider Just Two Levels

malloc.

```
int *pi
*p = 3;
```

```
int *pi
malloc(...) ← int
*p = 3;
```

Let's consider a 30-bit virtual address with 4kB page.

Single-Level Page Table

# of entries : 2^{18}.

size of page table : 2^{21}.

# of memory access ?

1 for 1 physical m.

Multi-Level Page Table 2 level.

size ?

1 page.

minimum : 2 pages (8 kB)

maximum : 2^{21} * 4kB

overhead 1 page (L1).

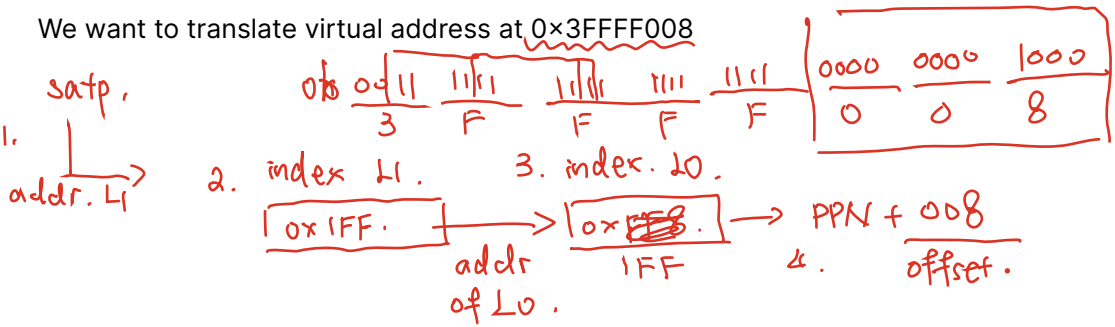
# of memory access.

3.

Consider Just Two Levels

Let's consider a 30-bit virtual address with 4kB page.

We want to translate virtual address at 0×3FFFF008



Consider Just Two Levels

Assume our process uses just one virtual address at `0x3FFFF008`
or `0b11_1111_1111_1111_1111_0000_0000_1000`
or `0b111111111_111111111_000000001000`

We'll just consider a 30-bit virtual address with a page size of 4096 bytes
We would need a 2 MiB page table if we only had one ($2^{18} \times 2^3$)

Instead, we have a 4 KiB L1 page table ($2^9 \times 2^3$) and a 4 KiB L0 page table
Total of 8 KiB instead of 2 MiB

Note: worst case if we used all virtual addresses we would consume 2 MiB + 4 KiB

Translating `3FFFF008` with 2 Page Tables

Consider the L1 table with the entry:

Index	PPN
511	0x8

Consider the L0 table located at `0x8000` with the entry:

Index	PPN
511	0xCAFE

The final translated physical address would be: `0xCAFE008`

Practice Question

Consider a system with 4096-byte pages, a PTE size of 6 bytes, 42-bit virtual addresses, and 48-bit physical addresses.

(a) How many offset bits are there in the virtual address?

(b) How many bits of the virtual address are left over for index bits?

(c) For a single page table, how large would the table need to be?

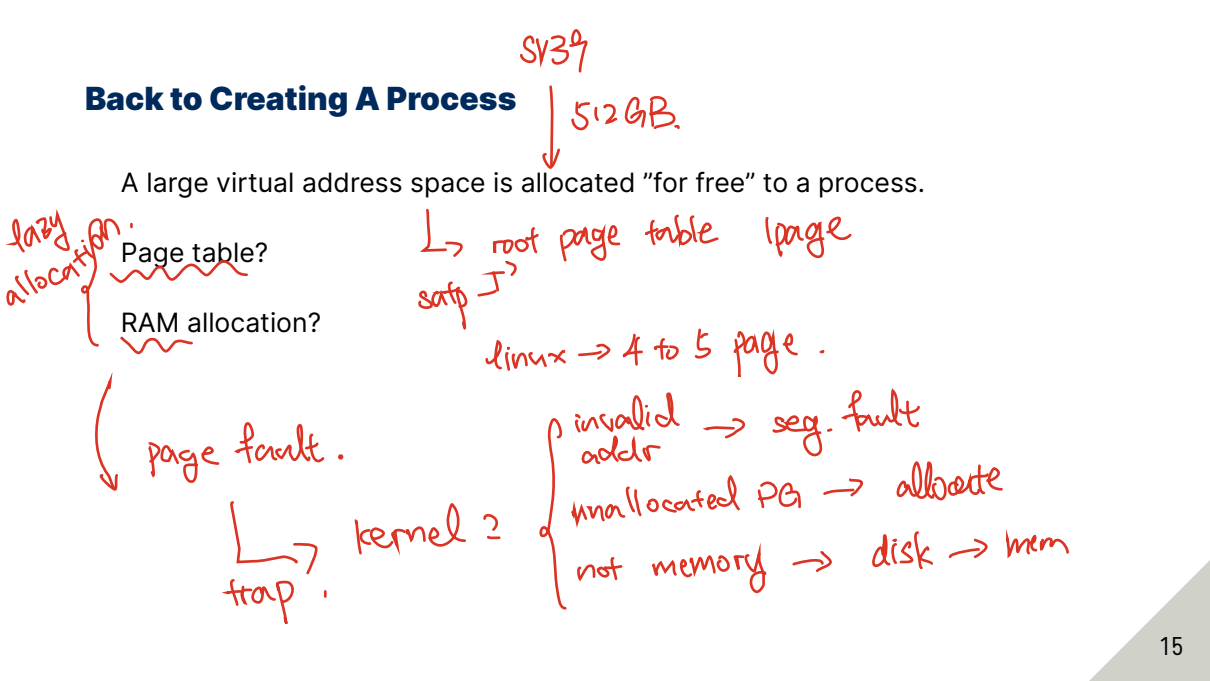
(d) How many PTEs could fit on a single page?

(e) How many levels of page tables would you need to ensure each page table fits on a single page?

Back to Creating A Process

A large virtual address space is allocated "for free" to a process.

- Page table?
- RAM allocation?



Back to Forking A Process

Copy-on-write ← . child & parent share PM.

Page table? → OS. allocs copy I

RAM allocation?

→ no extra.

why do we need to copy the page tables?

We don't! There's a system call for that — `vfork`

`vfork` shares all memory with the parent
It's undefined behavior to modify anything

Only used in very performance sensitive programs

Back to `malloc` and `free`

What happens when you `malloc` memory?

If a process "shrinks" its heap at the application level (e.g., via free() calls), does that actually reduce the process's overall memory usage from the operating system's perspective?

Alignment: Memory Eventually Lines Up with Byte 0

12 bits →

If pages are *4096 byte aligned* in memory is means pages always start when the lower 12 bits are zero, in computing we like alignment

If a page started at address `0x7C00` its last byte would be at address `0x8BFF`

Instead, a page would start at `0x7000` and end at `0x7FFF`

Question: Is address `0xEC` 8 byte aligned?

↳ last 3 bits to be 0.

Alignment Example

```
struct AlignmentDemo
{
    char char_a;
    int int_a;
    char char_b;
    char char_c;
    char char_d;
    int int_b;
    int int_c;
};
```

size (AlignmentDemo) = ?

char_a → offset 0

int_a → offset 4 (4-7).

char_b: 8

char_c: 9

char_d: 10

int_b → 12.

int_c → 16 (16-19)

char 1 byte.

int 4 bytes. → 4 byte aligned.

20.

Using the Page Tables for Every Memory Access is Slow

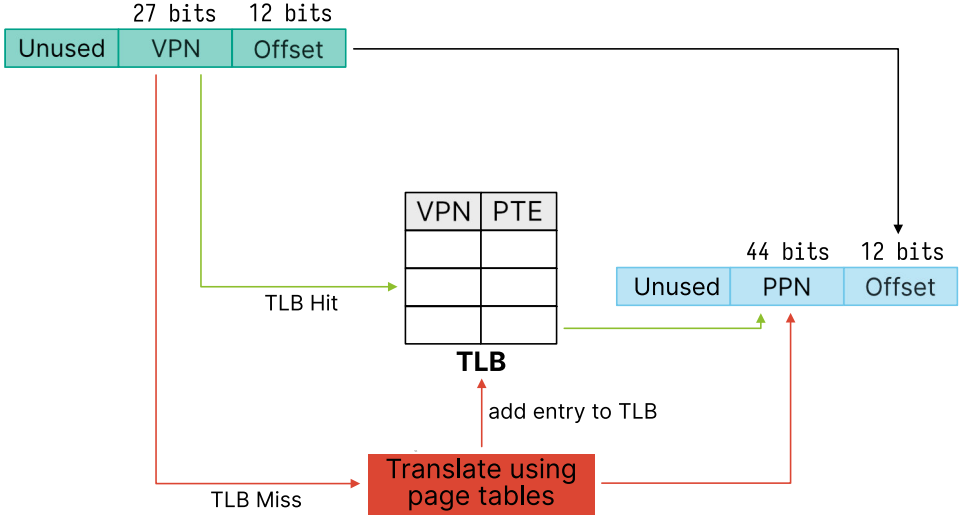
We need to follow pointers across multiple levels of page tables!

We'll likely access the same page multiple times (close to the first access time)

A process may only need a few VPN → PPN mappings at a time

Our solution is another computer science classic: caching

A Translation Look-Aside Buffer (TLB) Caches PTEs



Effective Access Time (EAT)

Assume a single page table
(there's only one additional memory access in the page table)

TLB_Hit_Time $=$ TLB_Search $+$ Mem
TLB_Miss_Time $=$ TLB_Search $+ 2 \times$ Mem
EAT $= \alpha \times$ TLB_Hit_Time $+ (1 - \alpha) \times$ TLB_Miss_Time

If $\alpha = 0.8$, TLB_Search $=$ 10 ns, and accesses take 100 ns, calculate EAT

Context Switches Require Handling the TLB

You can either flush the cache, or attach a process ID to the TLB

Most implementation just flush the TLB
  RISC-V uses a `sfence.vma` instruction to flush the TLB

On x86 loading the base page table will also flush the TLB